GUIA DE ESTUDO

Construção de instâncias e avaliação computacional do PAPT

Roteiro prático para sair de um JSON reduzido, organizar cenários progressivos e medir tempo de execução, GAP e qualidade da solução usando Julia, JuMP e CPLEX.

ETAPA	PERGUNTA A RESPONDER
1. Construir	Os dados representam um cenário coerente e documentado?
2. Validar	O modelo lê os dados e respeita as restrições esperadas?
3. Experimentar	Qual critério alterado afeta a viabilidade, o tempo e a solução?
4. Escalar	Como o esforço computacional muda à medida que a instância cresce?
5. Registrar	Outra pessoa consegue repetir e interpretar o experimento?

Princípio central: cada instância deve ser uma entrada versionada e identificável; o código matemático, os parâmetros do solver e o protocolo de teste devem permanecer controlados enquanto você compara os resultados.

Base de trabalho: integração JSON do PAPT em model_ic/PAPT_JuMP.jl, gerador sintético gerar_instancias.py e documentação oficial do JuMP/CPLEX consultada para as métricas e limites de execução.

1. O que é uma instância experimental?

Uma instância é um conjunto concreto de dados que, junto com uma versão da formulação matemática, define um problema específico a resolver. No PAPT, ela contém professores, turmas, disciplinas, carga horária, slots de aula, turnos, qualificação, disponibilidade e preferências.

Separe três objetos que não devem ser confundidos:

OBJETO	O QUE MUDA	EXEMPLO
Instância (JSON)	Dados do caso.	Mais turmas; menos professores disponíveis; outra distribuição de preferências.
Modelo (Julia)	Variáveis, restrições e função objetivo.	Alterar uma restrição de conflito ou mudar objetivo ponderado para lexicográfico.
Configuração experimental	Condições sob as quais o solver é executado.	Limite de 300 s; tolerância de GAP; número de threads.

Se mudar os dados, o modelo e os parâmetros do solver ao mesmo tempo, fica difícil atribuir a diferença observada a uma causa. Por isso, mantenha esses três objetos registrados separadamente.

Indicadores já presentes na sua formulação

SÍMBOLO / CAMPO	SIGNIFICADO NO EXPERIMENTO
|P|, |T|, |K|	Quantidade de professores, turmas e disciplinas.
|S|, |R|	Quantidade de slots distintos (dia-horário) e de turnos contabilizados pelo modelo.
e(i,j)	Elegibilidade efetiva; na integração atual, 1 quando qualificação e disponibilidade são verdadeiras.
w(i,j)	Preferência professor-turma usada na função objetivo.
F	Preferência total das alocações selecionadas.
s_min	Piso mínimo de satisfação entre professores ativos.
C	Quantidade de combinações professor-turno ocupadas.
g_max	Maior número de disciplinas distintas atribuídas a um professor.
Z	Valor da função objetivo ponderada usada no arquivo Julia atual.

A versão de integração usa uma função objetivo ponderada. O documento da modelagem também descreve uma alternativa lexicográfica; não misture resultados dessas duas formulações na mesma série como se fossem execuções do mesmo modelo.

2. Construção e validação dos dados

Comece distinguindo instâncias reais, reduzidas e sintéticas. Todas podem ser úteis, mas respondem a perguntas diferentes.

TIPO	USO PRINCIPAL	CUIDADO
Reduzida de referência	Verificar carregador, índices, restrições e valores de referência.	Não extrapolar o tempo desta instância para o problema real.
Extraída de oferta real	Avaliar aderência ao caso UEFS.	Documentar fonte, semestre, tratamento de dados e suposições.
Sintética controlada	Isolar efeitos de escala, preferências, capacidade e elegibilidade.	Não apresentar dados artificiais como dados reais da UEFS.
Inviável de propósito	Testar se o status e os diagnósticos do solver são interpretados corretamente.	Separar da série principal de desempenho.

Contrato JSON esperado pelo carregador atual

```
{
  "id": "sintetica_v001",
  "descricao": "Cenario pequeno de teste",
  "turnos_semanais": ["Seg-mat", "Seg-vesp"],
  "professores": [
    {"id":"P01", "nome":"Professor 01",
     "carga_min":4, "carga_max":12, "afastado":false}
  ],
  "turmas": [
    {"id":"DISC01-T01", "disciplina":"DISC01", "horas":4,
     "slots":[{"dia":"Seg", "horario":"09:30", "turno":"Seg-mat"}]}
  ],
  "pares_professor_turma": [
    {"professor":"P01", "turma":"DISC01-T01",
     "qualificado":true, "disponivel":true, "preferencia":4}
  ]
}
```

Valide cada JSON antes de executar o solver

- IDs de professores, turmas e turnos são únicos e consistentes; cada referência aponta para um elemento existente.
- Cada turma tem disciplina, carga horária positiva e pelo menos um slot; dia e horário identificam o slot exato.
- Os pares professor-turma indicam qualificação, disponibilidade e preferência numa escala definida (neste modelo, 0 a 5).
- A soma das cargas e os limites mínimos/máximos dos professores não se contradizem trivialmente.
- Verifique manualmente conflitos de slots, professores afastados e turmas sem nenhum professor elegível.
- Para uma instância sintética de referência, guarde uma alocação-testemunha viável conhecida, mas não trate isso como garantia após editar o JSON.

Na integração atual, pares não listados são tratados como inelegíveis. Mesmo assim, para auditoria é melhor que os JSONs de experimento documentem todos os pares professor-turma, especialmente quando a instância veio de dados reais.

3. Como definir critérios e criar cenários comparáveis

A regra experimental mais importante é alterar um fator por vez quando a pergunta for causal: por exemplo, medir o efeito da preferência mantendo escala, slots, qualificação, disponibilidade e limites de carga idênticos.

CRITÉRIO	COMO VARIAR	O QUE OBSERVAR
Escala	Aumentar |T| e ajustar |P| e |K| em níveis documentados.	Tempo, GAP, viabilidade, variáveis/restrições.
Qualificação	Alterar a proporção de pares qualificados; manter disponibilidade fixa.	Turmas sem candidatos, viabilidade, tempo.
Disponibilidade	Reduzir a disponibilidade sem alterar a qualificação.	Restrição do espaço de soluções e inviabilidade.
Preferências	Comparar perfis baixa, moderada, uniforme, alta e polarizada.	F e Z; alocação; efeito do objetivo ponderado.
Capacidade	Comparar limites estritos, moderados e folgados.	Distribuição de carga, viabilidade e dificuldade.
Conflitos	Alterar distribuição/densidade dos slots sobrepostos.	Número de restrições de conflito ativas e tempo.
Afastamento	Marcar professores como afastados e rever a elegibilidade.	Viabilidade e carga redistribuída.

Sementes e pares de comparação

• Para comparar duas distribuições aleatórias, mantenha a mesma semente sempre que quiser reduzir a variação que não vem do critério investigado.

• Guarde a semente e os parâmetros no próprio JSON (por exemplo, em criterios_geracao) e registre o nome do script/versão usado para gerar a instância.

• Não presuma que mudar somente o rótulo do perfil alterou apenas preferências: confirme os JSONs lado a lado e compare os campos que deveriam permanecer iguais.

Atenção à interpretação de “conflitos” no gerador

No gerador fornecido, a opção baixo/medio/alto muda o universo de horários candidatos; ela não define diretamente uma taxa empírica de conflitos. A quantidade de conflitos resultante depende também do número de turmas, slots por turma e regras de distribuição. Meça e reporte o resultado real, não apenas o rótulo do parâmetro.

4. Gerenciar as instâncias progressivamente

Mantenha uma pequena suíte de referência antes de produzir dezenas de arquivos. Cada fase deve ter uma pergunta objetiva e uma condição de saída.

FASE	O QUE FAZER	CONDIÇÃO PARA AVANÇAR
A. Referência	Executar reduzida_base.json; conferir cobertura, elegibilidade, carga, conflitos e componentes F, s_min, C e g_max.	Resultados consistentes com a referência documentada e restrições verificadas.
B. Testes de esquema	JSON válido, IDs duplicados, campo ausente, par não elegível e turma sem candidato.	Erros de dados são detectados; não são confundidos com status do solver.
C. Sintéticas pequenas	Poucos professores e turmas, mantendo a alocação-testemunha.	Resultado viável e interpretável; todos os componentes registrados.
D. Um fator por vez	Mudar somente preferências, disponibilidade, capacidade ou slots.	Diferença entre casos explicável pelo fator e pelos dados efetivamente alterados.
E. Crescimento	Aumentar turmas em degraus; manter protocolo e configurações fixos.	Tendência de tempo/GAP sem erro de entrada ou divergência de formulação.
F. Caso real	Converter dados reais para o mesmo contrato JSON; identificar origem e suposições.	A instância passa validação estrutural e revisão das regras semânticas.

Convenção de nome recomendada

```
instancias/
  ref_reduzida_v001.json
  sint_t12_p06_pref_uniforme_seed42_v001.json
  sint_t12_p06_pref_alta_seed42_v001.json
  sint_t24_p08_qual_060_seed42_v001.json
  real_uefs_2026_2_colengciv_v001.json

resultados/
  2026-10-25_configA_resumo.csv
  2026-10-25_configA/
    ref_reduzida_v001_resultado.json
    ref_reduzida_v001.lp
```

Regras de gestão

- Não sobrescreva um JSON que já foi usado em experimento; gere uma nova versão ou copie para uma pasta de arquivo.
- Separe instancias/ de resultados/. Não use arquivos de saída para substituir os dados de entrada.
- Inclua metadados: ID, descrição, tipo (real/sintética/reduzida), data, semente e parâmetros de geração.
- Mantenha o modelo Julia e os pesos da função objetivo fixos durante uma bateria; se mudar o modelo, registre outra versão experimental.
- Faça backup/versionamento dos dados e CSVs junto com o código. Um resultado sem versão da instância não é reprodutível.

5. Medir tempo usando somente Julia

Use tempos separados. Um único número não explica se o custo está na leitura do JSON, construção do modelo, otimização ou escrita dos arquivos.

MÉTRICA	O QUE INCLUI	USO
T_leitura	Ler e interpretar JSON, validar dados e derivar conjuntos/parâmetros.	Diagnosticar custo do carregador.
T_modelagem	Criar variáveis, expressões e restrições JuMP.	Medir construção do modelo, sem o solve.
T_wall_solve	Tempo de parede em torno de optimize!(model), via @elapsed.	Medir quanto o usuário espera pelo solver naquela chamada.
T_solver	Tempo relatado pelo otimizador, via solve_time(model) quando suportado.	Comparar com T_wall_solve; varia por solver.
T_total	Execução completa, incluindo entrada, modelo, solver e gravação.	Medir custo total do pipeline.

Exemplo para medir leitura, construção e otimização

```
using JuMP, CPLEX, JSON

# Pressupoe que carregar_instancia e construir_modelo estao definidos.
function medir_execucao(caminho; limite_s=300.0, gap_alvo=1e-4)
    t_leitura = @elapsed dados = carregar_instancia(caminho)
    t_modelagem = @elapsed refs = construir_modelo(dados)
    model = refs.model

    # Limite de parede para uma chamada de otimizacao.
    set_time_limit_sec(model, limite_s)
    # Tolerancia relativa de GAP do CPLEX; 1e-4 equivale a 0,01%.
    set_attribute(model, "CPX_PARAM_EPGAP", gap_alvo)

    t_wall_solve = @elapsed optimize!(model)
    t_solver = try
        solve_time(model)
    catch
        missing
    end

    return (; dados, refs, status=termination_status(model),
        tem_solucao=has_values(model), t_leitura, t_modelagem,
        t_wall_solve, t_solver, limite_s, gap_alvo)
end
```

Como usar o resultado

• T_wall_solve inclui o tempo de parede gasto na chamada optimize!, podendo incluir overhead da interface e preparação interna. T_solver é o tempo reportado pelo solver. Registre ambos em vez de presumir que sempre coincidem.

• T_leitura e T_modelagem precisam ser medidos fora do bloco de otimização. Se usar @elapsed em todo o resolver_instancia, esse total não pode ser rotulado como “tempo do CPLEX”.

• A primeira chamada em Julia pode incluir compilação JIT. Faça uma execução de aquecimento com uma instância pequena e não a use como medição principal.

• Repita cada cenário pelo menos 3 vezes, se o custo permitir, nas mesmas condições. Registre todas as execuções e reporte mediana e faixa; não apague uma execução lenta sem justificar.

Configuração experimental recomendada

Para comparação, fixe o limite de tempo, tolerância de GAP, número de threads, versão de Julia/CPLEX, máquina e estado de carga do computador. Não execute outras tarefas pesadas em paralelo. Para um primeiro protocolo, use por exemplo 300 s e GAP-alvo 0,01%; ajuste esses valores de forma justificada para os seus testes.

6. Entender e registrar o GAP

Em um problema de otimização inteira mista, o solver mantém uma solução incumbente (quando encontra uma solução viável) e um limite matemático que ajuda a provar o quão perto essa solução está do ótimo. O GAP relativo compara esses valores segundo a definição adotada pelo solver.

Para o modelo de maximização do PAPT, a solução incumbente é um valor viável de Z e o limite global é um limite superior de Z. Em termos intuitivos, quanto menor o GAP relativo, menor a diferença relativa ainda não fechada entre a melhor solução conhecida e o limite. A fórmula e os detalhes de arredondamento são definidos pelo solver; prefira registrar o GAP informado pelo CPLEX.

Três números que devem ser guardados juntos

CAMPO	INTERPRETAÇÃO
Objetivo incumbente (Z)	Valor da melhor solução viável conhecida, se houver.
Limite (bound)	Para maximização, limite superior usado para certificar a qualidade.
GAP relativo reportado	Estimativa percentual de distância relativa entre incumbente e limite, segundo o solver.
Status de término	Indica se houve ótimo provado, limite de tempo, inviabilidade ou outra condição.

Código para coletar objetivo, bound, GAP e status

```
function ou_missing(f)
    try
        return f()
    catch
        return missing
    end
end

function resumir_modelo(model)
    tem_solucao = has_values(model)
    return (
        status = string(termination_status(model)),
        primal_status = string(primal_status(model)),
        tem_solucao = tem_solucao,
        objetivo = tem_solucao ? objective_value(model) : missing,
        bound = ou_missing(() -> objective_bound(model)),
        gap_relativo = ou_missing(() -> relative_gap(model)),
        tempo_solver_s = ou_missing(() -> solve_time(model)),
    )
end
```

Não confunda o alvo de GAP com o GAP obtido

• gap_alvo = 1e-4 é uma configuração de parada equivalente a 0,01% (como fração, multiplique por 100 para obter percentagem).

• gap_relativo no resumo é o valor que o solver reporta ao terminar; pode ser maior do que o alvo se a execução parar por limite de tempo ou outra condição.

• Se status for TIME_LIMIT e houver solução viável, registre a incumbente e o GAP atual. Isso é uma solução encontrada, mas não prova otimalidade.

• Se não houver incumbente, deixe objetivo como ausente e não invente GAP. Investigue inviabilidade, limites e logs antes de classificar o caso.

• Se status for OPTIMAL, registre ainda assim tempo, objetivo, bound e GAP reportado. O significado operacional de “ótimo” depende das tolerâncias numéricas e critérios de parada.

A fórmula de GAP é dependente do solver e pode ser sensível quando o objetivo está próximo de zero. Não calcule um GAP manualmente e o misture ao GAP reportado sem identificar qual definição foi usada.

7. Tabela de testes: o que registrar

Use um registro por execução, não apenas uma linha por instância. Isso permite observar a variação entre repetições, limites de tempo e diferentes configurações.

GRUPO	COLUNAS SUGERIDAS
Identificação	id_instancia, arquivo_json, versao_json, tipo_instancia, semente, data_execucao
Dimensão	n_professores, n_turmas, n_disciplinas, n_slots, n_pares, densidade_elegibilidade
Configuração	versao_modelo, versao_julia, versao_cplex, threads, limite_tempo_s, gap_alvo
Desempenho	tempo_leitura_s, tempo_modelagem_s, tempo_wall_solve_s, tempo_solver_s, tempo_total_s
Resultado	status, tem_solucao, objetivo_Z, bound, gap_relativo, F, s_min, C, g_max
Análise	observacoes, motivo_da_falha, caminho_log, repeticao

Modelo de tabela para o relatório ou planilha

Instância	|P|	|T|	Status	Tempo solve (s)	Z	Bound	GAP (%)	F	s_min	C	g_max
ref_reduzida	3	4	OPTIMAL								
sint_t12_seed42	6	12									
sint_t24_seed42	8	24									
sint_t48_seed42	12	48									
real_uefs_v001											

Convenções: registre tempos em segundos; escolha e documente se o GAP é guardado como fração (por exemplo 0,0001) ou percentual (0,01%). A tabela acima sugere percentual para leitura humana. Não misture as duas escalas.

Exemplo de linha (ilustrativo, não medido)

Uma linha ilustrativa pode ter status TIME_LIMIT, Z preenchido, bound preenchido e GAP 2,4%. Isso significaria que o solver encontrou uma solução viável, mas encerrou o solve ao atingir o limite de tempo, sem fechar o GAP desejado. Não use valores fictícios como resultados de experimento; esta é apenas a interpretação do formato.

O CSV companion ajuda a começar o registro; o PDF contém a estrutura e as definições de cada coluna.

8. Protocolo de experimento e interpretação

Antes da bateria principal, defina um protocolo escrito e mantenha-o estável. Uma comparação confiável não depende apenas de executar os modelos: depende de que as condições sejam comparáveis.

DECISÃO	PROTOCOLO RECOMENDADO
Solução de referência	Use a instância reduzida e valide a alocação, a carga, conflitos, F, s_min, C e g_max.
Limite de tempo	Escolha limites progressivos (ex.: 60 s no teste inicial; 300 s ou mais em casos maiores) e declare-os antes da bateria.
Tolerância de GAP	Escolha um valor de parada e mantenha-o fixo na mesma série. Pode-se testar tolerâncias diferentes em uma série separada.
Repetições	Faça aquecimento; depois 3-5 repetições para casos rápidos ou ao menos repita os cenários mais importantes. Reporte mediana e amplitude.
Threads e ambiente	Fixe o número de threads, versão das bibliotecas e máquina; registre mudanças.
Códigos de falha	Separe dados inválidos, inviabilidade matemática, falta de solução incumbente, limite de tempo e erro de instalação/licença.

Ordem de análise

- Validade: a entrada passou na validação e o resultado respeita cada restrição?
- Viabilidade: o solver encontrou solução? Se não, é inviabilidade comprovada ou só falta de incumbente dentro do tempo?
- Qualidade: qual incumbente, bound, GAP e decomposição de F, s_min, C e g_max?
- Desempenho: quanto custam leitura, modelagem, solve e execução completa?
- Escala: como o tempo e o GAP mudam com |T|, |P|, densidade de elegibilidade e conflitos?

Cuidados específicos da função objetivo ponderada atual

A função objetivo atual combina componentes de escalas e sentidos diferentes: maximiza preferência total e piso de satisfação, enquanto penaliza turnos ocupados e o maior número de disciplinas por professor. Como os termos não são normalizados, os pesos influenciam a solução e o valor de Z. Não interprete Z maior em uma instância maior como melhoria direta; compare Z dentro do mesmo modelo e relate os componentes separadamente.

Validações que vale automatizar

- Cobertura: cada turma tem exatamente um professor atribuído.
- Elegibilidade: nenhum par escolhido pode ter e(i,j)=0.
- Carga: a carga atribuída a cada professor ativo está dentro dos limites registrados.
- Conflitos: nenhum professor está em duas turmas no mesmo dia e horário.
- Consistência de indicadores: F, s_min, C e g_max são recalculados a partir da solução e comparados com os valores do resumo.

9. Como incorporar métricas no seu Julia principal

No arquivo atual, a rotina de resolução já usa @elapsed em torno de optimize! e grava componentes do objetivo, mas ainda não registra de forma completa o bound, o GAP, o tempo de modelagem e o tempo do solver reportado. O trecho abaixo mostra um padrão para acrescentar esses campos ao resumo; adapte aos nomes efetivos das suas funções.

```
# No início do arquivo:
using MathOptInterface
const MOI = MathOptInterface

# Dentro de resolver_instancia, separar etapas:
t_leitura = @elapsed d = carregar_instancia(caminho)
t_modelagem = @elapsed refs = construir_modelo(d)
model = refs.model

set_time_limit_sec(model, 300.0)
set_attribute(model, "CPX_PARAM_EPGAP", 1e-4)

t_wall_solve = @elapsed optimize!(model)
t_solver = try solve_time(model) catch; missing end
status = string(termination_status(model))
tem_solucao = has_values(model)

objetivo = tem_solucao ? objective_value(model) : missing
bound = try objective_bound(model) catch; missing end
gap = try relative_gap(model) catch; missing end

# Incluir estes campos no Dict do resultado:
"tempo_leitura_s" => t_leitura,
"tempo_modelagem_s" => t_modelagem,
"tempo_wall_solve_s" => t_wall_solve,
"tempo_solver_s" => t_solver,
"limite_tempo_s" => 300.0,
"gap_alvo" => 1e-4,
"objetivo" => objetivo,
"bound" => bound,
"gap_relativo" => gap,
"tem_solucao" => tem_solucao
```

Notas de integração

• O exemplo ilustra os campos a registrar; a sintaxe de um Dict Julia exige que as entradas estejam dentro do Dict(...) e os campos não sejam duplicados. Ajuste o bloco à organização atual de resolver_instancia.

• Use um tratamento consistente para valores ausentes ao gravar JSON/CSV. Se o formato de JSON escolhido não aceitar missing diretamente, converta valores ausentes para nothing/null ou string vazia no CSV.

• Mantenha os logs do CPLEX habilitados durante a fase de validação; eles ajudam a distinguir “sem solução viável” de “tempo esgotado depois de encontrar incumbente”.

• Se quiser medir T_total, coloque um timer externo à função completa, incluindo leitura e gravação. Não some tempos parciais sem verificar se há etapas sobrepostas.

Execução sugerida

```
# Na pasta model_ic, no PowerShell:
julia --project=. PAPT_JuMP.jl reduzida_base.json
julia --project=. PAPT_JuMP.jl sintetica_pequena.json
julia --project=. PAPT_JuMP.jl --all
```

Use primeiro uma instância por chamada. Confira o status e os arquivos de saída; só depois execute --all, para que um erro seja fácil de localizar.

10. Checklist antes de fechar a tabela

☐ Cada instância tem ID único, JSON validado e origem/critério documentados.

☐ A instância de referência é resolvida e as restrições são verificadas independentemente do objetivo.

☐ Os parâmetros de geração e a semente estão preservados.

☐ A versão do modelo, função objetivo e pesos estão registrados.

☐ O número de threads, limite de tempo e GAP-alvo estão iguais dentro da mesma bateria.

☐ A primeira execução de Julia foi tratada como aquecimento quando o objetivo é medir o desempenho do solve.

☐ Tempos de leitura, construção, solve de parede e solve reportado estão em campos separados.

☐ Objetivo, bound, GAP e status foram registrados juntos; objetivo/GAP ausentes não foram preenchidos por estimativa.

☐ Resultados de casos inviáveis foram separados da série de desempenho normal.

☐ Os resultados foram exportados e versionados para não perder dados ao regenerar uma instância.

Perguntas para a discussão dos resultados

• O crescimento do tempo acompanha principalmente o número de turmas, o número de pares professor-turma ou a estrutura dos conflitos?

• Qual critério gera casos inviáveis com maior frequência? É um resultado esperado ou indica erro de dados?

• Com o limite de tempo fixo, quais instâncias encerram com ótimo provado e quais terminam com GAP residual?

• A melhoria em F vem acompanhada de piora em s_min, C ou g_max? Isso revela o efeito dos pesos escolhidos?

• A instância sintética é adequada para testar desempenho ou está artificialmente fácil por possuir uma alocação-testemunha bem distribuída?

Documentação técnica consultada

[1] JuMP documentation, “Models” - define set_time_limit_sec para limitar o tempo do solver: jump.dev/JuMP.jl/stable/manual/models/.

[2] JuMP API - atributos para tempo de solução, limite e GAP relativo: jump.dev/JuMP.jl/stable/api/JuMP/.

[3] CPLEX.jl - parâmetros do CPLEX são configurados por set_attribute e usam nomes de parâmetros da API C: github.com/jump-dev/CPLEX.jl.

[4] IBM ILOG CPLEX - limite de tempo do otimizador e critérios de término: IBM CPLEX optimizer time limit; GAP relativo: CPLEX Parameters Reference.

Este guia é um roteiro de estudo. Os limites e tamanhos indicados são exemplos iniciais para organizar seus testes, não resultados experimentais nem garantias de tempo. A execução precisa ser feita no seu ambiente Julia/CPLEX e registrada com as versões efetivamente usadas.